

```

//+-----
//|                                     RectangleZoneAnalyzer_07.mq4
//|
//|  Блок 07: формирование окончательного набора торговых паттернов
//|
//|  Вход : pattern_true_walkforward_summary.csv
//|
//|  Выход: final_patterns.csv
//|          final_patterns_local.csv
//|          final_patterns_structural.csv
//|          final_patterns_rejected.csv
//|
//|  В окончательный набор допускаются только паттерны, которые:
//|  1) получили OOSGrade = CONFIRMED_TWO_FOLDS;
//|  2) имеют OOSAccepted = YES;
//|  3) прошли оба независимых OOS-fold;
//|  4) дали положительный эффект во всех пригодных fold
//|      отдельно для локального или структурного результата.
//|
//|  Один исходный паттерн может создать две строки:
//|  LOCAL_REVERSAL и STRUCTURAL_REVERSAL.
//+-----
#property copyright "Copyright 2026"
#property version  "1.00"
#property strict
#property script_show_inputs

//--- Файлы находятся в MQL4\Files
input string InputSummaryFileName =
"pattern_true_walkforward_summary.csv";

```

```

input string OutputFinalPatternsFileName           =
"final_patterns.csv";
input string OutputLocalPatternsFileName           =
"final_patterns_local.csv";
input string OutputStructuralPatternsFileName      =
"final_patterns_structural.csv";
input string OutputRejectedFileName                =
"final_patterns_rejected.csv";

//--- При true берутся только Symbol() и Period() текущего графика
input bool    FilterCurrentChartOnly               = true;

//--- Обязательное подтверждение настоящего walk-forward
input string RequiredOOSGrade                      =
"CONFIRMED_TWO_FOLDS";
input int     RequiredPassedFolds                  = 2;
input double  RequiredPassSharePct                 = 100.0;

//--- Общая минимальная наполненность правила
input int     MinRuleOOSSamples                    = 50;
input int     StrongRuleSamples                    = 300;

//--- Отбор локального правила
input int     MinLocalDecisive                     = 50;
input double  MinLocalImprovementPctPoints         = 0.0;
input double  MinAvgLocalImprovementPctPoints     = 0.0;
input double  MinLocalWilsonLower95               = 0.0;
input double  MinLocalMfeMaeRatio                  = 0.0;

//--- Отбор структурного правила
input int     MinStructuralDecisive                 = 50;
input double  MinStructuralImprovementPctPoints    = 0.0;
input double  MinAvgStructuralImprovementPctPoints = 0.0;
input double  MinStructuralWilsonLower95           = 0.0;

//--- Границы торговых сессий должны совпадать с блоком 03
input int     AsiaEndHour                          = 7;
input int     LondonEndHour                        = 13;
input int     NewYorkEndHour                       = 21;

#define SUMMARY_CSV_COLUMNS 39

//+-----

//| Строка итогового walk-forward summary

//+-----

struct SummaryRow
{
    string patternID;

```

```

string patternType;
string ageBasis;
string symbol;
string timeframe;
string zoneType;
string ageBucket;
string touchBucket;
string depthBucket;
string approachBucket;
string zoneHeightBucket;
string sessionBucket;

int      selectedFolds;
int      eligibleFolds;
int      passedFolds;
double   passSharePct;

int      localEligibleFolds;
int      localPositiveFolds;
double   avgLocalImprovement;

int      structuralEligibleFolds;
int      structuralPositiveFolds;
double   avgStructuralImprovement;

int      pooledSamples;

int      pooledLocalDecisive;
double   pooledLocalReversalPct;
double   pooledParentLocalReversalPct;
double   pooledLocalImprovement;

int      pooledStructuralDecisive;
double   pooledStructuralOppositePct;
double   pooledParentStructuralOppositePct;
double   pooledStructuralImprovement;

double   pooledAvgMFE;
double   pooledAvgMAE;
double   pooledMfeMaeRatio;

int      lastSelectedFold;
string   lastFoldEligible;
string   lastFoldPass;
string   oosGrade;
string   oosAccepted;
};

```

//+-----

//| Окончательное торговое правило

```
//+-----
struct FinalRule
{
    int sourceIndex;

    string ruleType;
    string finalRuleID;
    string signalDirection;
    string targetMode;
    string ruleGrade;

    int overallRank;
    int typeRank;

    int positiveFolds;
    int eligibleFolds;
    int decisiveSamples;

    double successPct;
    double wilsonLower95;
    double parentSuccessPct;
    double improvementPctPoints;
    double avgFoldImprovementPctPoints;
    double score;
};
```

```
//+-----

//| Причина исключения
```

```
//+-----
struct RejectedRow
{
    int sourceIndex;
    string scope;
    string reason;
};
```

```
SummaryRow g_rows[];
FinalRule g_rules[];
RejectedRow g_rejected[];
```

```
//--- Индексы колонок summary
int c_patternID = -1;
int c_patternType = -1;
int c_ageBasis = -1;
int c_symbol = -1;
int c_timeframe = -1;
int c_zoneType = -1;
int c_ageBucket = -1;
```

```

int c_touchBucket           = -1;
int c_depthBucket          = -1;
int c_approachBucket        = -1;
int c_zoneHeightBucket      = -1;
int c_sessionBucket         = -1;
int c_selectedFolds         = -1;
int c_eligibleFolds         = -1;
int c_passedFolds           = -1;
int c_passSharePct          = -1;
int c_localEligibleFolds    = -1;
int c_localPositiveFolds    = -1;
int c_avgLocalImprovement   = -1;
int c_structuralEligibleFolds = -1;
int c_structuralPositiveFolds = -1;
int c_avgStructuralImprovement = -1;
int c_pooledSamples         = -1;
int c_pooledLocalDecisive   = -1;
int c_pooledLocalReversalPct = -1;
int c_pooledParentLocalReversalPct = -1;
int c_pooledLocalImprovement = -1;
int c_pooledStructuralDecisive = -1;
int c_pooledStructuralOppositePct = -1;
int c_pooledParentStructuralOppositePct = -1;
int c_pooledStructuralImprovement = -1;
int c_pooledAvgMFE          = -1;
int c_pooledAvgMAE          = -1;
int c_pooledMfeMaeRatio     = -1;
int c_lastSelectedFold      = -1;
int c_lastFoldEligible      = -1;
int c_lastFoldPass          = -1;
int c_oosGrade              = -1;
int c_oosAccepted           = -1;

```

```

int g_totalInputRows        = 0;
int g_otherChartRows        = 0;
int g_confirmedPatterns     = 0;
int g_localRules            = 0;
int g_structuralRules        = 0;

```

```
//+-----
```

```
//| Имя текущего таймфрейма
```

```
//+-----
```

```

string TimeframeToString(int timeframe)
{
    switch(timeframe)
    {
        case PERIOD_M1:    return("M1");
        case PERIOD_M5:    return("M5");
        case PERIOD_M15:   return("M15");
    }
}

```

```

        case PERIOD_M30: return("M30");
        case PERIOD_H1:  return("H1");
        case PERIOD_H4:  return("H4");
        case PERIOD_D1:  return("D1");
        case PERIOD_W1:  return("W1");
        case PERIOD_MN1: return("MN1");
    }

    return(IntegerToString(timeframe));
}

//+-----
//| Удаление BOM
//+-----
string StripBOM(string value)
{
    if(StringLen(value) > 0 && StringGetCharacter(value, 0) ==
65279)
        return(StringSubstr(value, 1));

    if(StringLen(value) >= 3 &&
        StringGetCharacter(value, 0) == 239 &&
        StringGetCharacter(value, 1) == 187 &&
        StringGetCharacter(value, 2) == 191)
    {
        return(StringSubstr(value, 3));
    }

    return(value);
}

//+-----
//| Поиск колонки
//+-----
int FindColumn(string &headers[], string columnName)
{
    int count = ArraySize(headers);

    for(int i = 0; i < count; i++)
    {
        string current = headers[i];

        if(i == 0)
            current = StripBOM(current);

        if(current == columnName)

```

```

        return(i);
    }

    return(-1);
}

//+-----
//| Чтение фиксированной CSV-записи
//+-----
bool ReadCsvRecord(int handle, int columns, string &fields[])
{
    ArrayResize(fields, columns);

    if(FileIsEnding(handle))
        return(false);

    for(int i = 0; i < columns; i++)
    {
        if(FileIsEnding(handle) && i == 0)
            return(false);

        fields[i] = FileReadString(handle);
    }

    return(StringLen(fields[0]) > 0);
}

//+-----
//| Преобразования
//+-----
int ParseInt(string value, int defaultValue = 0)
{
    if(StringLen(value) == 0)
        return(defaultValue);

    return((int)StringToInteger(value));
}

double ParseDouble(string value, double defaultValue = 0.0)
{
    if(StringLen(value) == 0)
        return(defaultValue);

    return(StringToDouble(value));
}

```

```

string YesNo(bool value)
{
    return(value ? "YES" : "NO");
}

//+-----

//| Карта колонок summary

//+-----
bool MapSummaryColumns(string &headers[])
{
    c_patternID = FindColumn(headers,
"PatternID");
    c_patternType = FindColumn(headers,
"PatternType");
    c_ageBasis = FindColumn(headers,
"AgeBasis");
    c_symbol = FindColumn(headers,
"Symbol");
    c_timeframe = FindColumn(headers,
"Timeframe");
    c_zoneType = FindColumn(headers,
"ZoneType");
    c_ageBucket = FindColumn(headers,
"AgeBucket");
    c_touchBucket = FindColumn(headers,
"TouchBucket");
    c_depthBucket = FindColumn(headers,
"DepthBucket");
    c_approachBucket = FindColumn(headers,
"ApproachBucket");
    c_zoneHeightBucket = FindColumn(headers,
"ZoneHeightBucket");
    c_sessionBucket = FindColumn(headers,
"SessionBucket");
    c_selectedFolds = FindColumn(headers,
"SelectedFolds");
    c_eligibleFolds = FindColumn(headers,
"EligibleFolds");
    c_passedFolds = FindColumn(headers,
"PassedFolds");
    c_passSharePct = FindColumn(headers,
"PassSharePct");
    c_localEligibleFolds = FindColumn(headers,
"LocalEligibleFolds");
    c_localPositiveFolds = FindColumn(headers,
"LocalPositiveFolds");
    c_avgLocalImprovement = FindColumn(headers,
"AvgLocalImprovementPctPoints");
}

```

```

    c_structuralEligibleFolds           = FindColumn(headers,
"StructuralEligibleFolds");
    c_structuralPositiveFolds           = FindColumn(headers,
"StructuralPositiveFolds");
    c_avgStructuralImprovement          = FindColumn(headers,
"AvgStructuralImprovementPctPoints");
    c_pooledSamples                     = FindColumn(headers,
"PooledOOSSamples");
    c_pooledLocalDecisive               = FindColumn(headers,
"PooledOOSLocalDecisive");
    c_pooledLocalReversalPct           = FindColumn(headers,
"PooledOOSLocalReversalPct");
    c_pooledParentLocalReversalPct     = FindColumn(headers,
"PooledParentLocalReversalPct");
    c_pooledLocalImprovement           = FindColumn(headers,
"PooledLocalImprovementPctPoints");
    c_pooledStructuralDecisive         = FindColumn(headers,
"PooledOOSStructuralDecisive");
    c_pooledStructuralOppositePct      = FindColumn(headers,
"PooledOOSStructuralOppositePct");
    c_pooledParentStructuralOppositePct = FindColumn(headers,
"PooledParentStructuralOppositePct");
    c_pooledStructuralImprovement      = FindColumn(headers,
"PooledStructuralImprovementPctPoints");
    c_pooledAvgMFE                     = FindColumn(headers,
"PooledAvgMFE_CloseATR");
    c_pooledAvgMAE                     = FindColumn(headers,
"PooledAvgMAE_CloseATR");
    c_pooledMfeMaeRatio                = FindColumn(headers,
"PooledMFE_MAE_Ratio");
    c_lastSelectedFold                 = FindColumn(headers,
"LastSelectedFold");
    c_lastFoldEligible                 = FindColumn(headers,
"LastFoldEligible");
    c_lastFoldPass                     = FindColumn(headers,
"LastFoldPass");
    c_oosGrade                         = FindColumn(headers,
"OOSGrade");
    c_oosAccepted                      = FindColumn(headers,
"OOSAccepted");

    if(c_patternID < 0 || c_patternType < 0 || c_ageBasis < 0 ||
        c_symbol < 0 || c_timeframe < 0 || c_zoneType < 0 ||
        c_ageBucket < 0 || c_touchBucket < 0 || c_depthBucket < 0
    ||
        c_approachBucket < 0 || c_zoneHeightBucket < 0 ||
        c_sessionBucket < 0 || c_selectedFolds < 0 ||
        c_eligibleFolds < 0 || c_passedFolds < 0 ||
        c_passSharePct < 0 || c_localEligibleFolds < 0 ||
        c_localPositiveFolds < 0 || c_avgLocalImprovement < 0 ||
        c_structuralEligibleFolds < 0 ||
c_structuralPositiveFolds < 0 ||

```

```

        c_avgStructuralImprovement < 0 || c_pooledSamples < 0 ||
        c_pooledLocalDecisive < 0 || c_pooledLocalReversalPct < 0
    ||
        c_pooledParentLocalReversalPct < 0 ||
c_pooledLocalImprovement < 0 ||
        c_pooledStructuralDecisive < 0 ||
c_pooledStructuralOppositePct < 0 ||
        c_pooledParentStructuralOppositePct < 0 ||
        c_pooledStructuralImprovement < 0 || c_pooledAvgMFE < 0
    ||
        c_pooledAvgMAE < 0 || c_pooledMfeMaeRatio < 0 ||
        c_lastSelectedFold < 0 || c_lastFoldEligible < 0 ||
        c_lastFoldPass < 0 || c_oosGrade < 0 || c_oosAccepted < 0
)
{
    Print("Ошибка: в ", InputSummaryFileName,
        " отсутствуют обязательные колонки блока 07.");
    return(false);
}

return(true);
}

//+-----

//| Добавление строки summary

//+-----
void AppendSummaryRow(string &fields[])
{
    int index = ArraySize(g_rows);
    ArrayResize(g_rows, index + 1);

    g_rows[index].patternID           = fields[c_patternID];
    g_rows[index].patternType         = fields[c_patternType];
    g_rows[index].ageBasis             = fields[c_ageBasis];
    g_rows[index].symbol              = fields[c_symbol];
    g_rows[index].timeframe            = fields[c_timeframe];
    g_rows[index].zoneType             = fields[c_zoneType];
    g_rows[index].ageBucket            = fields[c_ageBucket];
    g_rows[index].touchBucket          = fields[c_touchBucket];
    g_rows[index].depthBucket          = fields[c_depthBucket];
    g_rows[index].approachBucket       = fields[c_approachBucket];
    g_rows[index].zoneHeightBucket     = fields[c_zoneHeightBucket];
};
    g_rows[index].sessionBucket        = fields[c_sessionBucket];

    g_rows[index].selectedFolds        = ParseInt(fields[
c_selectedFolds]);
    g_rows[index].eligibleFolds        = ParseInt(fields[
c_eligibleFolds]);

```

```

    g_rows[index].passedFolds      = ParseInt(fields[
c_passedFolds]);
    g_rows[index].passSharePct     = ParseDouble(fields[
c_passSharePct]);

    g_rows[index].localEligibleFolds = ParseInt(fields[
c_localEligibleFolds]);
    g_rows[index].localPositiveFolds = ParseInt(fields[
c_localPositiveFolds]);
    g_rows[index].avgLocalImprovement = ParseDouble(fields[
c_avgLocalImprovement]);

    g_rows[index].structuralEligibleFolds =
        ParseInt(fields[c_structuralEligibleFolds]);
    g_rows[index].structuralPositiveFolds =
        ParseInt(fields[c_structuralPositiveFolds]);
    g_rows[index].avgStructuralImprovement =
        ParseDouble(fields[c_avgStructuralImprovement]);

    g_rows[index].pooledSamples = ParseInt(fields[
c_pooledSamples]);

    g_rows[index].pooledLocalDecisive =
        ParseInt(fields[c_pooledLocalDecisive]);
    g_rows[index].pooledLocalReversalPct =
        ParseDouble(fields[c_pooledLocalReversalPct]);
    g_rows[index].pooledParentLocalReversalPct =
        ParseDouble(fields[c_pooledParentLocalReversalPct]);
    g_rows[index].pooledLocalImprovement =
        ParseDouble(fields[c_pooledLocalImprovement]);

    g_rows[index].pooledStructuralDecisive =
        ParseInt(fields[c_pooledStructuralDecisive]);
    g_rows[index].pooledStructuralOppositePct =
        ParseDouble(fields[c_pooledStructuralOppositePct]);
    g_rows[index].pooledParentStructuralOppositePct =
        ParseDouble(fields[c_pooledParentStructuralOppositePct]);
    g_rows[index].pooledStructuralImprovement =
        ParseDouble(fields[c_pooledStructuralImprovement]);

    g_rows[index].pooledAvgMFE = ParseDouble(fields[
c_pooledAvgMFE], -1.0);
    g_rows[index].pooledAvgMAE = ParseDouble(fields[
c_pooledAvgMAE], -1.0);
    g_rows[index].pooledMfeMaeRatio =
        ParseDouble(fields[c_pooledMfeMaeRatio], -1.0);

    g_rows[index].lastSelectedFold = ParseInt(fields[
c_lastSelectedFold]);
    g_rows[index].lastFoldEligible = fields[c_lastFoldEligible];
    g_rows[index].lastFoldPass      = fields[c_lastFoldPass];
    g_rows[index].oosGrade          = fields[c_oosGrade];

```

```

    g_rows[index].oosAccepted      = fields[c_oosAccepted];
}

//+-----

//| Загрузка итогов блока 06

//+-----
bool LoadSummary()
{
    ResetLastError();

    int handle = FileOpen(
        InputSummaryFileName,
        FILE_READ | FILE_CSV | FILE_ANSI,
        ';'
    );

    if(handle == INVALID_HANDLE)
    {
        Print("Ошибка открытия ", InputSummaryFileName,
            ". Код ошибки: ", GetLastError());
        ResetLastError();
        return(false);
    }

    string headers[];
    ArrayResize(headers, SUMMARY_CSV_COLUMNS);

    for(int i = 0; i < SUMMARY_CSV_COLUMNS; i++)
        headers[i] = FileReadString(handle);

    if(!MapSummaryColumns(headers))
    {
        FileClose(handle);
        return(false);
    }

    string fields[];
    string currentTF = TimeframeToString(Period());

    while(ReadCsvRecord(handle, SUMMARY_CSV_COLUMNS, fields))
    {
        g_totalInputRows++;

        if(FilterCurrentChartOnly &&
            (fields[c_symbol] != Symbol() || fields[c_timeframe]
!= currentTF))
        {
            g_otherChartRows++;
            continue;
        }
    }
}

```

```

    }

    AppendSummaryRow(fields);
}

FileClose(handle);

if(ArraySize(g_rows) == 0)
{
    Print("Ошибка: в ", InputSummaryFileName,
        " нет строк для ", Symbol(), " ", currentTF, ".");
    return(false);
}

return(true);
}

```

//+-----

//| Нижняя 95%-граница Уилсона

//+-----

```

double WilsonLower95(int decisiveSamples, double successPct)
{
    if(decisiveSamples <= 0)
        return(0.0);

    int successes =
        (int)MathRound((double)decisiveSamples * successPct /
100.0);

    if(successes < 0)
        successes = 0;

    if(successes > decisiveSamples)
        successes = decisiveSamples;

    double n = (double)decisiveSamples;
    double p = (double)successes / n;
    double z = 1.959963984540054;
    double z2 = z * z;
    double denominator = 1.0 + z2 / n;
    double center = p + z2 / (2.0 * n);
    double margin =
        z * MathSqrt((p * (1.0 - p) + z2 / (4.0 * n)) / n);

    double lower = (center - margin) / denominator;

    if(lower < 0.0)
        lower = 0.0;
}

```

```

    return(lower * 100.0);
}

//+-----

//| Добавление причины через разделитель
//+-----
void AddReason(string &reason, string text)
{
    if(StringLen(reason) > 0)
        reason += "|";

    reason += text;
}

//+-----

//| Добавление исключённой записи
//+-----
void AddRejected(int sourceIndex, string scope, string reason)
{
    int index = ArraySize(g_rejected);
    ArrayResize(g_rejected, index + 1);

    g_rejected[index].sourceIndex = sourceIndex;
    g_rejected[index].scope = scope;
    g_rejected[index].reason = reason;
}

//+-----

//| Проверка подтверждения паттерна
//+-----
bool IsConfirmedPattern(SummaryRow &row, string &reason)
{
    reason = "";

    if(row.oosAccepted != "YES")
        AddReason(reason, "OOS_ACCEPTED_NO");

    if(row.oosGrade != RequiredOOSGrade)
        AddReason(reason, "GRADE_NOT_" + RequiredOOSGrade);

    if(row.selectedFolds < RequiredPassedFolds)
        AddReason(reason, "SELECTED_FOLDS_LT_REQUIRED");
}

```

```

if(row.eligibleFolds < RequiredPassedFolds)
    AddReason(reason, "ELIGIBLE_FOLDS_LT_REQUIRED");

if(row.passedFolds < RequiredPassedFolds)
    AddReason(reason, "PASSED_FOLDS_LT_REQUIRED");

if(row.passSharePct + 0.000001 < RequiredPassSharePct)
    AddReason(reason, "PASS_SHARE_LT_REQUIRED");

if(row.lastFoldEligible != "YES")
    AddReason(reason, "LAST_FOLD_NOT_ELIGIBLE");

if(row.lastFoldPass != "YES")
    AddReason(reason, "LAST_FOLD_NOT_PASSED");

if(row.pooledSamples < MinRuleOOSamples)
    AddReason(reason, "OOS_SAMPLES_LT_MIN");

return(StringLen(reason) == 0);
}

//+-----

//| Проверка локального правила

//+-----
bool IsLocalRuleAccepted(SummaryRow &row, double wilson, string
&reason)
{
    reason = "";

    if(row.localEligibleFolds < RequiredPassedFolds)
        AddReason(reason, "LOCAL_ELIGIBLE_FOLDS_LT_REQUIRED");

    if(row.localPositiveFolds < row.localEligibleFolds)
        AddReason(reason, "LOCAL_NOT_POSITIVE_IN_ALL_FOLDS");

    if(row.pooledLocalDecisive < MinLocalDecisive)
        AddReason(reason, "LOCAL_DECISIVE_LT_MIN");

    if(row.pooledLocalImprovement <=
MinLocalImprovementPctPoints)
        AddReason(reason, "LOCAL_POOLED_EDGE_NOT_ABOVE_MIN");

    if(row.avgLocalImprovement <=
MinAvgLocalImprovementPctPoints)
        AddReason(reason, "LOCAL_AVG_EDGE_NOT_ABOVE_MIN");

    if(wilson + 0.000001 < MinLocalWilsonLower95)
        AddReason(reason, "LOCAL_WILSON_LT_MIN");

```

```

if (MinLocalMfeMaeRatio > 0.0 &&
    row.pooledMfeMaeRatio + 0.000001 < MinLocalMfeMaeRatio)
{
    AddReason(reason, "LOCAL_MFE_MAE_LT_MIN");
}

return(StringLen(reason) == 0);
}

//+-----

//| Проверка структурного правила

//+-----
bool IsStructuralRuleAccepted(
    SummaryRow &row,
    double wilson,
    string &reason
)
{
    reason = "";

    if (row.structuralEligibleFolds < RequiredPassedFolds)
        AddReason(reason, "STRUCTURAL_ELIGIBLE_FOLDS_LT_REQUIRED"
);

    if (row.structuralPositiveFolds < row.structuralEligibleFolds
)
        AddReason(reason, "STRUCTURAL_NOT_POSITIVE_IN_ALL_FOLDS"
);

    if (row.pooledStructuralDecisive < MinStructuralDecisive)
        AddReason(reason, "STRUCTURAL_DECISIVE_LT_MIN");

    if (row.pooledStructuralImprovement <=
MinStructuralImprovementPctPoints)
        AddReason(reason, "STRUCTURAL_POOLED_EDGE_NOT_ABOVE_MIN"
);

    if (row.avgStructuralImprovement <=
MinAvgStructuralImprovementPctPoints)
    {
        AddReason(reason, "STRUCTURAL_AVG_EDGE_NOT_ABOVE_MIN");
    }

    if (wilson + 0.000001 < MinStructuralWilsonLower95)
        AddReason(reason, "STRUCTURAL_WILSON_LT_MIN");

    return(StringLen(reason) == 0);
}

```

```

//+-----
//| Оценка окончательного правила
//+-----
double CalculateRuleScore(
    string ruleType,
    SummaryRow &row,
    int positiveFolds,
    int eligibleFolds,
    double successPct,
    double wilson,
    double improvement
)
{
    double sampleFactor = 0.0;

    if(StrongRuleSamples > 0)
    {
        sampleFactor =
            MathMin(1.0,
                    MathSqrt((double) row.pooledSamples /
                             (double) StrongRuleSamples));
    }

    double consistency = 0.0;

    if(eligibleFolds > 0)
        consistency = (double) positiveFolds / (double)
eligibleFolds;

    double payoffFactor = 0.0;

    if(ruleType == "LOCAL_REVERSAL" && row.pooledMfeMaeRatio >
0.0)
        payoffFactor = MathMin(2.0, row.pooledMfeMaeRatio) / 2.0;

    // Основной вес даётся нижней границе Уилсона и OOS-эффекту.
    return(
        wilson * 0.45 +
        successPct * 0.20 +
        MathMax(0.0, improvement) * 1.50 +
        sampleFactor * 10.0 +
        consistency * 5.0 +
        payoffFactor * 3.0
    );
}

//+-----

```

```
//| Категория правила
```

```
//+-----
```

```
string GetRuleGrade(  
    SummaryRow &row,  
    int decisiveSamples,  
    double improvement,  
    double wilson  
)  
{  
    if(row.pooledSamples >= StrongRuleSamples &&  
        decisiveSamples >= 200 &&  
        improvement >= 3.0)  
    {  
        return("A_STRONG_CONFIRMED");  
    }  
  
    if(row.pooledSamples >= 100 && decisiveSamples >= 80)  
        return("B_CONFIRMED");  
  
    if(wilson > 0.0)  
        return("C_CONFIRMED_SMALL");  
  
    return("D_CONFIRMED");  
}
```

```
//+-----
```

```
//| Добавление окончательного правила
```

```
//+-----
```

```
void AddFinalRule(  
    int sourceIndex,  
    string ruleType,  
    int positiveFolds,  
    int eligibleFolds,  
    int decisiveSamples,  
    double successPct,  
    double parentSuccessPct,  
    double improvement,  
    double avgFoldImprovement,  
    double wilson  
)  
{  
    SummaryRow row = g_rows[sourceIndex];  
  
    int index = ArraySize(g_rules);  
    ArrayResize(g_rules, index + 1);  
  
    g_rules[index].sourceIndex = sourceIndex;
```

```

g_rules[index].ruleType = ruleType;

if(ruleType == "LOCAL_REVERSAL")
{
    g_rules[index].finalRuleID = "FINAL_LOCAL_" +
row.patternID;
    g_rules[index].targetMode = "PRIMARY_REACTION_DISTANCE";
    g_localRules++;
}
else
{
    g_rules[index].finalRuleID = "FINAL_STRUCTURAL_" +
row.patternID;
    g_rules[index].targetMode = "NEXT_OPPOSITE_ZONE";
    g_structuralRules++;
}

g_rules[index].signalDirection =
    row.zoneType == "BULL" ? "BUY" : "SELL";

g_rules[index].ruleGrade =
    GetRuleGrade(row, decisiveSamples, improvement, wilson);

g_rules[index].overallRank = 0;
g_rules[index].typeRank = 0;
g_rules[index].positiveFolds = positiveFolds;
g_rules[index].eligibleFolds = eligibleFolds;
g_rules[index].decisiveSamples = decisiveSamples;
g_rules[index].successPct = successPct;
g_rules[index].wilsonLower95 = wilson;
g_rules[index].parentSuccessPct = parentSuccessPct;
g_rules[index].improvementPctPoints = improvement;
g_rules[index].avgFoldImprovementPctPoints =
avgFoldImprovement;
g_rules[index].score =
    CalculateRuleScore(
        ruleType,
        row,
        positiveFolds,
        eligibleFolds,
        successPct,
        wilson,
        improvement
    );
}

```

//+-----

//| Построение окончательного набора

//+-----

```

void BuildFinalRules()
{
    for(int i = 0; i < ArraySize(g_rows); i++)
    {
        SummaryRow row = g_rows[i];
        string patternReason = "";

        if(!IsConfirmedPattern(row, patternReason))
        {
            AddRejected(i, "PATTERN", patternReason);
            continue;
        }

        g_confirmedPatterns++;

        double localWilson =
            WilsonLower95(
                row.pooledLocalDecisive,
                row.pooledLocalReversalPct
            );

        string localReason = "";

        if(IsLocalRuleAccepted(row, localWilson, localReason))
        {
            AddFinalRule(
                i,
                "LOCAL_REVERSAL",
                row.localPositiveFolds,
                row.localEligibleFolds,
                row.pooledLocalDecisive,
                row.pooledLocalReversalPct,
                row.pooledParentLocalReversalPct,
                row.pooledLocalImprovement,
                row.avgLocalImprovement,
                localWilson
            );
        }
        else
        {
            AddRejected(i, "LOCAL_RULE", localReason);
        }

        double structuralWilson =
            WilsonLower95(
                row.pooledStructuralDecisive,
                row.pooledStructuralOppositePct
            );

        string structuralReason = "";
    }
}

```

```

        if(IsStructuralRuleAccepted(row, structuralWilson,
structuralReason))
        {
            AddFinalRule(
                i,
                "STRUCTURAL_REVERSAL",
                row.structuralPositiveFolds,
                row.structuralEligibleFolds,
                row.pooledStructuralDecisive,
                row.pooledStructuralOppositePct,
                row.pooledParentStructuralOppositePct,
                row.pooledStructuralImprovement,
                row.avgStructuralImprovement,
                structuralWilson
            );
        }
    else
    {
        AddRejected(i, "STRUCTURAL_RULE", structuralReason);
    }
}

//+-----

//| Сортировка по итоговому score

//+-----
void SortRules()
{
    int count = ArraySize(g_rules);

    for(int i = 0; i < count - 1; i++)
    {
        int best = i;

        for(int j = i + 1; j < count; j++)
        {
            bool better = false;

            if(g_rules[j].score > g_rules[best].score + 0.0000001)
                better = true;
            else if(MathAbs(g_rules[j].score - g_rules[best].score
) <= 0.0000001)
            {
                SummaryRow rowJ = g_rows[g_rules[j].sourceIndex];
                SummaryRow rowB = g_rows[g_rules[best].sourceIndex
];

                if(rowJ.pooledSamples > rowB.pooledSamples)
                    better = true;
            }
        }
    }
}

```

```

        else if(rowJ.pooledSamples == rowB.pooledSamples &&
                g_rules[j].improvementPctPoints >
                g_rules[best].improvementPctPoints)
        {
            better = true;
        }
    }

    if(better)
        best = j;
}

if(best != i)
{
    FinalRule temp = g_rules[i];
    g_rules[i] = g_rules[best];
    g_rules[best] = temp;
}
}

int localRank = 0;
int structuralRank = 0;

for(int k = 0; k < count; k++)
{
    g_rules[k].overallRank = k + 1;

    if(g_rules[k].ruleType == "LOCAL_REVERSAL")
    {
        localRank++;
        g_rules[k].typeRank = localRank;
    }
    else
    {
        structuralRank++;
        g_rules[k].typeRank = structuralRank;
    }
}
}

//+-----

//| Декодирование возрастного диапазона

//+-----
bool GetAgeBounds(string bucket, double &minValue, double &
maxValue)
{
    minValue = 0.0;
    maxValue = -1.0;

```

```

        if(bucket == "00_0_15_MIN")           { minValue = 0.0;
maxValue = 15.0;   return(true); }
        if(bucket == "01_15_30_MIN")           { minValue = 15.0;
maxValue = 30.0;   return(true); }
        if(bucket == "02_30_60_MIN")           { minValue = 30.0;
maxValue = 60.0;   return(true); }
        if(bucket == "03_1_2_HOURS")           { minValue = 60.0;
maxValue = 120.0;  return(true); }
        if(bucket == "04_2_4_HOURS")           { minValue = 120.0;
maxValue = 240.0;  return(true); }
        if(bucket == "05_4_8_HOURS")           { minValue = 240.0;
maxValue = 480.0;  return(true); }
        if(bucket == "06_8_24_HOURS")          { minValue = 480.0;
maxValue = 1440.0; return(true); }
        if(bucket == "07_24_48_HOURS")         { minValue = 1440.0;
maxValue = 2880.0; return(true); }
        if(bucket == "08_48_HOURS_PLUS")       { minValue = 2880.0;
maxValue = -1.0;   return(true); }

    return(false);
}

```

//+-----

//| Декодирование номера касания

//+-----

```

bool GetTouchBounds(string bucket, int &minValue, int &maxValue
)
{
    minValue = -1;
    maxValue = -1;

    if(bucket == "TOUCH_1")           { minValue = 1; maxValue = 1;
return(true); }
    if(bucket == "TOUCH_2")           { minValue = 2; maxValue = 2;
return(true); }
    if(bucket == "TOUCH_3")           { minValue = 3; maxValue = 3;
return(true); }
    if(bucket == "TOUCH_4")           { minValue = 4; maxValue = 4;
return(true); }
    if(bucket == "TOUCH_5_PLUS")      { minValue = 5; maxValue = -1;
return(true); }

    return(false);
}

```

//+-----

//| Декодирование глубины

```
//+-----  
bool GetDepthBounds(string bucket, double &minValue, double &  
maxValue)  
{  
    minValue = -1.0;  
    maxValue = -1.0;  
  
    if(bucket == "DEPTH_0_25_PCT")    { minValue = 0.0;  
maxValue = 25.0; return(true); }  
    if(bucket == "DEPTH_25_50_PCT")    { minValue = 25.0;  
maxValue = 50.0; return(true); }  
    if(bucket == "DEPTH_50_75_PCT")    { minValue = 50.0;  
maxValue = 75.0; return(true); }  
    if(bucket == "DEPTH_75_100_PCT")    { minValue = 75.0;  
maxValue = 100.0; return(true); }  
  
    return(false);  
}
```

```
//+-----  
  
//| Декодирование скорости подхода
```

```
//+-----  
bool GetApproachBounds(string bucket, double &minValue, double  
&maxValue)  
{  
    minValue = -1.0;  
    maxValue = -1.0;  
  
    if(bucket == "APPROACH_FLAT_OR_AWAY")  
    {  
        minValue = -1.0;  
        maxValue = 0.0;  
        return(true);  
    }  
  
    if(bucket == "APPROACH_SLOW_LT_0_5_ATR")  
    {  
        minValue = 0.0;  
        maxValue = 0.5;  
        return(true);  
    }  
  
    if(bucket == "APPROACH_MEDIUM_0_5_1_ATR")  
    {  
        minValue = 0.5;  
        maxValue = 1.0;  
        return(true);  
    }  
}
```

```

if(bucket == "APPROACH_FAST_GE_1_ATR")
{
    minValue = 1.0;
    maxValue = -1.0;
    return(true);
}

return(false);
}

//+-----

//| Декодирование высоты зоны

//+-----
bool GetHeightBounds(string bucket, double &minValue, double &
maxValue)
{
    minValue = -1.0;
    maxValue = -1.0;

    if(bucket == "HEIGHT_LT_0_25_ATR")
    {
        minValue = 0.0;
        maxValue = 0.25;
        return(true);
    }

    if(bucket == "HEIGHT_0_25_0_50_ATR")
    {
        minValue = 0.25;
        maxValue = 0.50;
        return(true);
    }

    if(bucket == "HEIGHT_0_50_0_75_ATR")
    {
        minValue = 0.50;
        maxValue = 0.75;
        return(true);
    }

    if(bucket == "HEIGHT_0_75_1_00_ATR")
    {
        minValue = 0.75;
        maxValue = 1.00;
        return(true);
    }

    if(bucket == "HEIGHT_GE_1_00_ATR")
    {

```

```

        minValue = 1.00;
        maxValue = -1.0;
        return(true);
    }

    return(false);
}

//+-----
//| Декодирование серверной сессии
//+-----
bool GetSessionBounds(string bucket, int &startHour, int &
endHour)
{
    startHour = -1;
    endHour = -1;

    if(bucket == "ASIA_SERVER_TIME")
    {
        startHour = 0;
        endHour = AsiaEndHour;
        return(true);
    }

    if(bucket == "LONDON_SERVER_TIME")
    {
        startHour = AsiaEndHour;
        endHour = LondonEndHour;
        return(true);
    }

    if(bucket == "NEW_YORK_SERVER_TIME")
    {
        startHour = LondonEndHour;
        endHour = NewYorkEndHour;
        return(true);
    }

    if(bucket == "OTHER_SERVER_TIME")
    {
        startHour = NewYorkEndHour;
        endHour = 24;
        return(true);
    }

    return(false);
}

```

```

//+-----
// | CSV-утилиты
//+-----
string EscapeCsv(string value)
{
    bool needQuotes =
        StringFind(value, ";") >= 0 ||
        StringFind(value, "\"") >= 0 ||
        StringFind(value, "\r") >= 0 ||
        StringFind(value, "\n") >= 0;

    if(!needQuotes)
        return(value);

    StringReplace(value, "\"", "\\\"");
    return("\"" + value + "\"");
}

void AppendField(string &line, string value)
{
    if(StringLen(line) > 0)
        line += ";";

    line += EscapeCsv(value);
}

string DoubleText(double value, int digits)
{
    return(DoubleToString(value, digits));
}

string OptionalDouble(bool enabled, double value, int digits)
{
    if(!enabled || value < 0.0)
        return("");

    return(DoubleToString(value, digits));
}

string OptionalInt(bool enabled, int value)
{
    if(!enabled || value < 0)
        return("");

    return(IntegerToString(value));
}

//+-----

```

```
///  
//| Общий заголовок файла правил
```

```
///  
-----  
string RulesHeader()  
{  
    return(  
        "OverallRank;TypeRank;FinalRuleID;Enabled;RuleType;" +  
        "SignalDirection;TargetMode;RuleGrade;PatternID;PatternType;" +  
        "AgeBasis;Symbol;Timeframe;ZoneType;AgeBucket;UseAgeFilter;" +  
        "AgeMinMinutes;AgeMaxMinutesExclusive;TouchBucket;" +  
        "UseTouchFilter;TouchMin;TouchMax;DepthBucket;UseDepthFilter;" +  
        "DepthMinPct;DepthMaxPctExclusive;ApproachBucket;" +  
        "UseApproachFilter;ApproachMinATR;ApproachMaxATRExclusive;" +  
        "ZoneHeightBucket;UseHeightFilter;HeightMinATR;" +  
        "HeightMaxATRExclusive;SessionBucket;UseSessionFilter;" +  
        "SessionStartHour;SessionEndHourExclusive;SelectedFolds;" +  
        "PassedFolds;PositiveFolds;EligibleFolds;PooledOOSamples;" +  
        "PooledDecisive;OOSuccessPct;WilsonLower95;ParentSuccessPct;" +  
        "ImprovementPctPoints;AvgFoldImprovementPctPoints;" +  
        "AvgMFE_CloseATR;AvgMAE_CloseATR;MFE_MAE_Ratio;RuleScore;" +  
        "OOSGrade;OOSAccepted\r\n"  
    );  
}
```

```
///  
-----  
//| Формирование строки окончательного правила
```

```
///  
-----  
string BuildRuleLine(FinalRule &rule)  
{  
    SummaryRow row = g_rows[rule.sourceIndex];  
  
    double ageMin = -1.0;  
    double ageMax = -1.0;  
    bool useAge = GetAgeBounds(row.ageBucket, ageMin, ageMax);  
  
    int touchMin = -1;  
    int touchMax = -1;  
    bool useTouch = GetTouchBounds(row.touchBucket, touchMin,  
touchMax);
```

```

double depthMin = -1.0;
double depthMax = -1.0;
bool useDepth = GetDepthBounds(row.depthBucket, depthMin,
depthMax);

double approachMin = -1.0;
double approachMax = -1.0;
bool useApproach =
    GetApproachBounds(row.approachBucket, approachMin,
approachMax);

double heightMin = -1.0;
double heightMax = -1.0;
bool useHeight =
    GetHeightBounds(row.zoneHeightBucket, heightMin,
heightMax);

int sessionStart = -1;
int sessionEnd = -1;
bool useSession =
    GetSessionBounds(row.sessionBucket, sessionStart,
sessionEnd);

string line = "";

AppendField(line, IntegerToString(rule.overallRank));
AppendField(line, IntegerToString(rule.typeRank));
AppendField(line, rule.finalRuleID);
AppendField(line, "YES");
AppendField(line, rule.ruleType);
AppendField(line, rule.signalDirection);
AppendField(line, rule.targetMode);
AppendField(line, rule.ruleGrade);
AppendField(line, row.patternID);
AppendField(line, row.patternType);
AppendField(line, row.ageBasis);
AppendField(line, row.symbol);
AppendField(line, row.timeframe);
AppendField(line, row.zoneType);
AppendField(line, row.ageBucket);
AppendField(line, YesNo(useAge));
AppendField(line, OptionalDouble(useAge, ageMin, 2));
AppendField(line, OptionalDouble(useAge && ageMax >= 0.0,
ageMax, 2));
AppendField(line, row.touchBucket);
AppendField(line, YesNo(useTouch));
AppendField(line, OptionalInt(useTouch, touchMin));
AppendField(line, OptionalInt(useTouch && touchMax >= 0,
touchMax));
AppendField(line, row.depthBucket);
AppendField(line, YesNo(useDepth));
AppendField(line, OptionalDouble(useDepth, depthMin, 2));

```

```

    AppendField(line, OptionalDouble(useDepth && depthMax >= 0.0
, depthMax, 2));
    AppendField(line, row.approachBucket);
    AppendField(line, YesNo(useApproach));
    AppendField(line, OptionalDouble(useApproach, approachMin, 4
));
    AppendField(line, OptionalDouble(useApproach && approachMax
>= 0.0, approachMax, 4));
    AppendField(line, row.zoneHeightBucket);
    AppendField(line, YesNo(useHeight));
    AppendField(line, OptionalDouble(useHeight, heightMin, 4));
    AppendField(line, OptionalDouble(useHeight && heightMax >=
0.0, heightMax, 4));
    AppendField(line, row.sessionBucket);
    AppendField(line, YesNo(useSession));
    AppendField(line, OptionalInt(useSession, sessionStart));
    AppendField(line, OptionalInt(useSession, sessionEnd));
    AppendField(line, IntegerToString(row.selectedFolds));
    AppendField(line, IntegerToString(row.passedFolds));
    AppendField(line, IntegerToString(rule.positiveFolds));
    AppendField(line, IntegerToString(rule.eligibleFolds));
    AppendField(line, IntegerToString(row.pooledSamples));
    AppendField(line, IntegerToString(rule.decisiveSamples));
    AppendField(line, DoubleText(rule.successPct, 4));
    AppendField(line, DoubleText(rule.wilsonLower95, 6));
    AppendField(line, DoubleText(rule.parentSuccessPct, 4));
    AppendField(line, DoubleText(rule.improvementPctPoints, 4));
    AppendField(line, DoubleText(
rule.avgFoldImprovementPctPoints, 4));
    AppendField(line, DoubleText(row.pooledAvgMFE, 6));
    AppendField(line, DoubleText(row.pooledAvgMAE, 6));
    AppendField(line, DoubleText(row.pooledMfeMaeRatio, 6));
    AppendField(line, DoubleText(rule.score, 6));
    AppendField(line, row.oosGrade);
    AppendField(line, row.oosAccepted);

    return(line);
}

```

//+-----

//| Сохранение набора правил

//+-----

```

bool SaveRules(string fileName, string ruleTypeFilter)
{
    ResetLastError();

```

```

    int handle = FileOpen(fileName, FILE_WRITE | FILE_TXT |
FILE_ANSI);

```

```

if(handle == INVALID_HANDLE)
{
    Print("Ошибка создания ", fileName,
        ". Код ошибки: ", GetLastError());
    ResetLastError();
    return(false);
}

FileWriteString(handle, RulesHeader());

int saved = 0;

for(int i = 0; i < ArraySize(g_rules); i++)
{
    if(StringLen(ruleTypeFilter) > 0 &&
        g_rules[i].ruleType != ruleTypeFilter)
    {
        continue;
    }

    string line = BuildRuleLine(g_rules[i]);
    FileWriteString(handle, line + "\r\n");
    saved++;
}

FileClose(handle);

Print(fileName, ": записано правил = ", saved);
return(true);
}

```

//+-----

//| Сохранение отклонённых паттернов и направлений

//+-----

```

bool SaveRejected()
{
    ResetLastError();

    int handle = FileOpen(
        OutputRejectedFileName,
        FILE_WRITE | FILE_TXT | FILE_ANSI
    );

    if(handle == INVALID_HANDLE)
    {
        Print("Ошибка создания ", OutputRejectedFileName,
            ". Код ошибки: ", GetLastError());
        ResetLastError();
        return(false);
    }
}

```

```

    }

    string header =

"PatternID;RejectedScope;RejectReason;PatternType;AgeBasis;" +

"Symbol;Timeframe;ZoneType;AgeBucket;TouchBucket;DepthBucket;"
+

"ApproachBucket;ZoneHeightBucket;SessionBucket;SelectedFolds;"
+

"EligibleFolds;PassedFolds;PassSharePct;OOSGrade;OOSAccepted;"
+
    "PooledOOSSamples;LocalEligibleFolds;LocalPositiveFolds;"
+
    "PooledLocalDecisive;PooledLocalReversalPct;" +

"PooledParentLocalReversalPct;PooledLocalImprovementPctPoints;"
+
    "AvgLocalImprovementPctPoints;StructuralEligibleFolds;" +
    "StructuralPositiveFolds;PooledStructuralDecisive;" +

"PooledStructuralOppositePct;PooledParentStructuralOppositePct;"
+
    "PooledStructuralImprovementPctPoints;" +

"AvgStructuralImprovementPctPoints;PooledMFE_MAE_Ratio\r\n";

FileWriteString(handle, header);

for(int i = 0; i < ArraySize(g_rejected); i++)
{
    SummaryRow row = g_rows[g_rejected[i].sourceIndex];
    string line = "";

    AppendField(line, row.patternID);
    AppendField(line, g_rejected[i].scope);
    AppendField(line, g_rejected[i].reason);
    AppendField(line, row.patternType);
    AppendField(line, row.ageBasis);
    AppendField(line, row.symbol);
    AppendField(line, row.timeframe);
    AppendField(line, row.zoneType);
    AppendField(line, row.ageBucket);
    AppendField(line, row.touchBucket);
    AppendField(line, row.depthBucket);
    AppendField(line, row.approachBucket);
    AppendField(line, row.zoneHeightBucket);
    AppendField(line, row.sessionBucket);
    AppendField(line, IntegerToString(row.selectedFolds));
    AppendField(line, IntegerToString(row.eligibleFolds));
    AppendField(line, IntegerToString(row.passedFolds));
    AppendField(line, DoubleText(20, 6, row.passSharePct, 4));
}

```

```

        AppendField(line, IntegerToString(row.localEligibleFolds
));
        AppendField(line, IntegerToString(row.localPositiveFolds
));
        AppendField(line, IntegerToString(row.pooledLocalDecisive
));
        AppendField(line, DoubleText(row.pooledLocalReversalPct,
4));
        AppendField(line, DoubleText(
row.pooledParentLocalReversalPct, 4));
        AppendField(line, DoubleText(row.pooledLocalImprovement,
4));
        AppendField(line, DoubleText(row.avgLocalImprovement, 4
));
        AppendField(line, IntegerToString(
row.structuralEligibleFolds));
        AppendField(line, IntegerToString(
row.structuralPositiveFolds));
        AppendField(line, IntegerToString(
row.pooledStructuralDecisive));
        AppendField(line, DoubleText(
row.pooledStructuralOppositePct, 4));
        AppendField(line, DoubleText(
row.pooledParentStructuralOppositePct, 4));
        AppendField(line, DoubleText(
row.pooledStructuralImprovement, 4));
        AppendField(line, DoubleText(row.avgStructuralImprovement
, 4));
        AppendField(line, DoubleText(row.pooledMfeMaeRatio, 6));

        FileWriteString(handle, line + "\r\n");
    }

    FileClose(handle);

    Print(OutputRejectedFileName,
        ": записано исключений = ", ArraySize(g_rejected));

    return(true);
}

```

```

//+-----

//| Проверка входных параметров

//+-----
bool ValidateInputs()
{
    if(RequiredPassedFolds < 1)
    {

```

```

        Print(
"Ошибка: RequiredPassedFolds должен быть не меньше 1.");
        return(false);
    }

    if(RequiredPassSharePct < 0.0 || RequiredPassSharePct >
100.0)
    {
        Print(
"Ошибка: RequiredPassSharePct должен быть от 0 до 100.");
        return(false);
    }

    if(MinRuleOOSSamples < 1 || StrongRuleSamples < 1 ||
        MinLocalDecisive < 1 || MinStructuralDecisive < 1)
    {
        Print(
"Ошибка: минимальные размеры выборки должны быть больше 0.");
        return(false);
    }

    if(AsiaEndHour < 0 || AsiaEndHour > 24 ||
        LondonEndHour < 0 || LondonEndHour > 24 ||
        NewYorkEndHour < 0 || NewYorkEndHour > 24 ||
        AsiaEndHour > LondonEndHour || LondonEndHour >
NewYorkEndHour)
    {
        Print("Ошибка: неверно заданы границы торговых сессий.");
        return(false);
    }

    return(true);
}

```

//+-----

//| Главная функция скрипта

//+-----

```

void OnStart()
{
    Print(
"=====
");
    Print(
"RectangleZoneAnalyzer_07: окончательный набор паттернов");
    Print(
"=====
");

    ArrayResize(g_rows, 0);
    ArrayResize(g_rules, 0);
    ArrayResize(g_rejected, 0);
}

```

```

    if(!ValidateInputs())
        return;

    if(!LoadSummary())
        return;

    BuildFinalRules();
    SortRules();

    if(ArraySize(g_rules) == 0)
    {
        Print(
"Ошибка: ни одно окончательное правило не прошло фильтр.");
        SaveRejected();
        return;
    }

    bool savedAll = SaveRules(OutputFinalPatternsFileName, "");
    bool savedLocal = SaveRules(OutputLocalPatternsFileName,
"LOCAL_REVERSAL");
    bool savedStructural =
        SaveRules(OutputStructuralPatternsFileName,
"STRUCTURAL_REVERSAL");
    bool savedRejected = SaveRejected();

    Print(
"-----"
);
    Print("Строк во входном summary          : ", g_totalInputRows
);
    Print("Пропущено другого графика          : ", g_otherChartRows
);
    Print("Обработано строк текущего графика: ", ArraySize(
g_rows));
    Print("Подтверждено в двух OOS-fold      : ",
g_confirmedPatterns);
    Print("Локальных правил                    : ", g_localRules);
    Print("Структурных правил                    : ", g_structuralRules
);
    Print("Всего окончательных правил            : ", ArraySize(g_rules
));
    Print("Отклонённых записей                    : ", ArraySize(
g_rejected));
    Print(
"-----"
);

    if(savedAll && savedLocal && savedStructural &&
savedRejected)
        Print("Блок 07 завершён успешно.");
    else
        Print(
"Блок 07 завершён с ошибкой сохранения одного из файлов.");
}

```

// +-----